AI ENGINEERING INTERVIEW QUESTIONS PART 1

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. AI Engineering Foundations

1. What is AI Engineering ?

- AI Engineering is the process of designing, building, deploying and maintaining AI/ML based applications in real-world environments.
- It combines Data Engineering, Model Development, Software Engineering and MLOps to deliver scalable AI solutions.
- Focus is not just on model training, but on end-to-end lifecycle from data → model → deployment → monitoring → improvement.

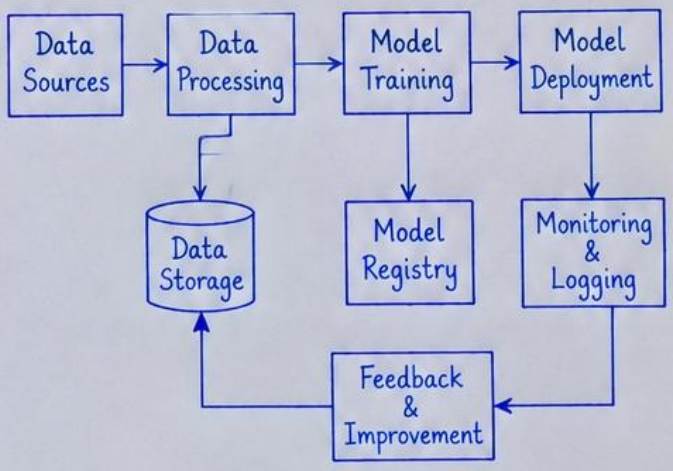
2. AI Engineering vs Data Science

Data Science	AI Engineering
Focus on analysis	Focus on production
Exploratory work	End-to-end system
Build models (not prod)	Build & deploy models
Primarily research	Scalable & maintainable
Uses notebooks	Uses software engineering
Evaluation offline	Monitoring in production
Small datasets	Large-scale real data

3. Key Components of AI Engineering

- Data Layer : Data collection, storage, processing
- Model Layer : Training, fine-tuning, evaluation
- Serving Layer : Deploying models as APIs
- Infrastructure : Compute, storage, scalability
- Monitoring : Performance, drift, logs, alerts
- Application Layer : User interface and integration

4. High Level Architecture



5. Common Use Cases

- Chatbots and virtual assistants (e.g., customer support)
- Document understanding (PDF, images, forms)
- Recommendation systems
- Search and information retrieval
- Image and video analysis
- Fraud detection
- Code generation and copilots
- Automation using intelligent agents

6. Important Skills for AI Engineers

- Programming : Python, SQL, APIs
- Machine Learning : Model training, evaluation
- Deep Learning : Neural networks, Transformers
- LLM Application Development : RAG, Agents, Tools
- Data Engineering : ETL, data pipelines
- MLOps : Deployment, monitoring, scaling
- System Design : Scalable AI architectures
- Cloud Platforms : AWS/GCP/Azure
- Domain Knowledge : Real-world problem solving

7. Interview Focus Areas

- Core ML/DL concepts
- LLM and Generative AI
- RAG, Vector Databases
- System design for AI applications
- Model deployment and MLOps
- Real-world problem solving and trade-offs

8. Key Takeaways

- AI Engineering is about building real-world solutions
- It requires a mix of ML, software engineering and MLOps
- Focus on scalability, reliability and monitoring
- Understand both theory and practical implementation
- Be ready to discuss real-world use cases and trade-offs

Rahul Yadav

2. Python For AI Engineering

1. Why Python for AI ?

- Simple and easy to learn syntax
- Rich ecosystem of AI/ML libraries
- Large community support
- Used in research as well as production
- Flexible and integrates well with tools
- Suitable for rapid prototyping and experimentation

2. Popular Libraries

Library	Purpose
NumPy	Numerical computing (arrays, matrices)
Pandas	Data analysis and manipulation
Matplotlib	Data visualization
Seaborn	Statistical visualization
Scikit-learn	Machine learning algorithms
PyTorch	Deep learning (research & production)
TensorFlow	Deep learning (big projects)
Hugging Face	Pretrained models, Transformers, NLP
LangChain	Build LLM applications
OpenAI	Access to OpenAI models (GPT etc.)

3. Basic Syntax

```
# Variables
name = "Rahul"
age = 28
print("Hello", name, "Age:", age)

# List
numbers = [1, 2, 3, 4]
print(numbers)

# Dictionary
person = {"name": "Rahul", "role": "AI Engineer"}
print(person["name"])
```

4. Working with Data (Pandas)

```
import pandas as pd

# Read CSV file
df = pd.read_csv("data.csv")
print(df.head())

# Basic operations
print(df.shape)          # rows, columns
print(df.info())         # data types
print(df.describe())     # statistical summary
```

5. NumPy for Numerical Computing

```
import numpy as np

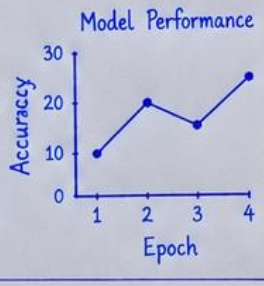
# Create array
arr = np.array([1, 2, 3, 4])
print(arr)

# Matrix operations
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(a + b)          # element-wise addition
print(np.dot(a, b))   # matrix multiplication
```

6. Data Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Line plot
plt.plot([1,2,3,4], [10,20,15,25])
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Model Performance")
plt.show()
```



7. Example: Simple ML Model (Scikit-learn)

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict
pred = model.predict([[5]])
print("Prediction:", pred[0])
```

8. Key Takeaways

- Python is the foundation language for AI engineering
- Learn core libraries: NumPy, Pandas, Matplotlib, Scikit-learn
- Use PyTorch or TensorFlow for deep learning projects
- Practice data preprocessing and visualization
- Build small projects to gain hands-on experience
- Understand real-world data handling and pipelines
- Python is essential for interviews, projects and production

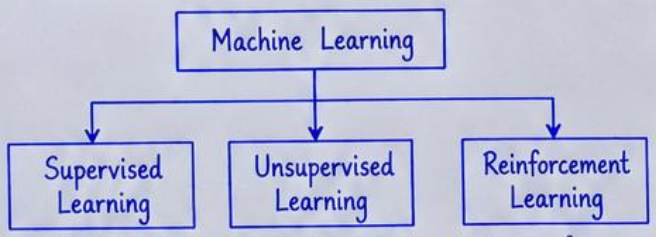
Rahul Yadav

4. Machine Learning Basics

1. What is Machine Learning ?

- Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn patterns from data without being explicitly programmed.
- It uses statistical techniques and algorithms to make predictions or decisions.
- ML improves its performance with more data and experience.

2. Types of Machine Learning



- Labeled data
- Predict output
- Classification
- Regression

- Unlabeled data
- Find hidden patterns
- Clustering
- Dimensionality reduction

- Learn from interaction
- Reward based
- Sequential decision making
- Examples: Games, Robotics

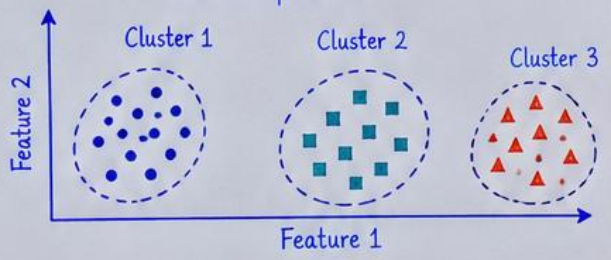
3. Supervised Learning

- Uses labeled data (input → output).
- Learns a mapping from input features to target output.
- Used for classification and regression.
- Examples: Spam detection, house price prediction, disease prediction.



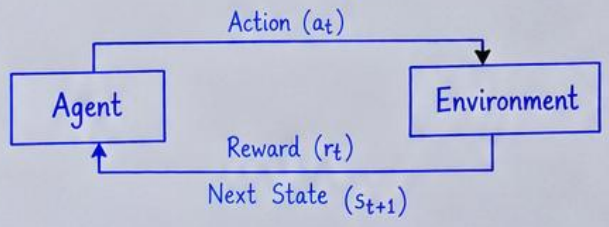
4. Unsupervised Learning

- Uses unlabeled data (only input).
- Finds hidden structure or patterns in data.
- Used for clustering, dimensionality reduction, anomaly detection, etc.
- Examples: Customer segmentation, topic modeling, data compression.



5. Reinforcement Learning

- Learns by interacting with an environment.
- Takes actions, receives rewards, and improves over time.
- Used for sequential decision making.
- Examples: Game playing (AlphaGo), robotics, autonomous systems.



6. Common ML Algorithms

Type	Algorithm	Use Case
Supervised	Linear Regression	Predict continuous value
	Logistic Regression	Binary classification
	Decision Trees	Classification / Regression
	Random Forest	Improved accuracy
	SVM	High dimensional data
Unsupervised	K-Means	Clustering
	Hierarchical Clustering	Cluster hierarchy
	PCA	Dimensionality reduction
	t-SNE	Data visualization

7. Key Evaluation Metrics

Task	Metrics
Classification	Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC-AUC
Regression	MAE, MSE, RMSE, R^2 Score
Clustering	Silhouette Score, Davies-Bouldin Index
Ranking / Recommendation	Precision@K, Recall@K, NDCG

8. Key Takeaways

- ML is about learning from data, not hardcoding rules.
- Choose the right type of learning based on the problem.
- Data quality is very important.
- Start with simple models and baseline metrics.
- Evaluate using appropriate metrics.
- Practice with real datasets and projects.
- Understand the trade-offs of different algorithms.

Rahul Yadav

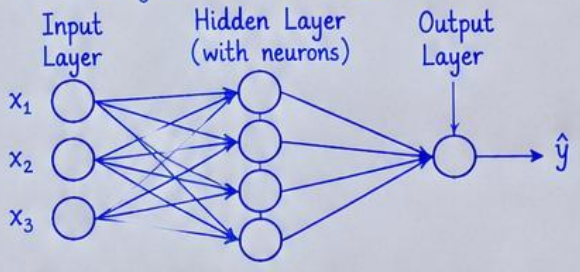
5. Deep Learning Essentials

1. What is Deep Learning?

- Deep Learning is a subset of Machine Learning that uses neural networks with multiple layers to learn complex patterns from data.
- It automatically learns hierarchical representations from raw data (images, text, audio, etc).
- It has achieved state-of-the-art results in computer vision, NLP, speech and more.

2. Artificial Neural Network (ANN)

- A neural network is inspired by the human brain.
- It consists of layers of interconnected neurons that learn from data.
- Each neuron performs a weighted sum of inputs followed by an activation function.



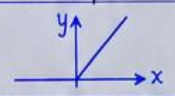
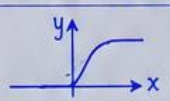
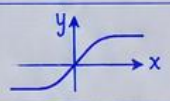
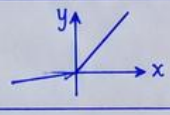
3. Neuron Computation

- Each neuron takes multiple inputs, multiplies them with weights, adds a bias and applies an activation function.

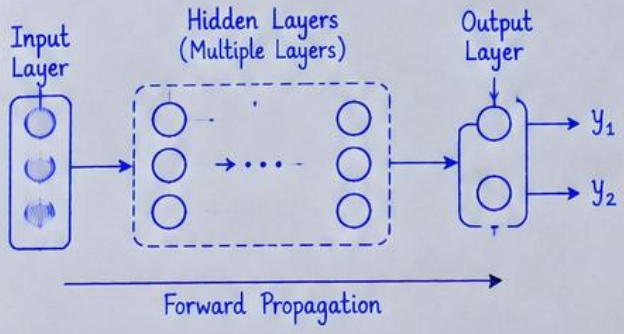
$$z = \sum (w_i x_i) + b \rightarrow a = f(z)$$

- x_i : input features
- w_i : weights
- b : bias
- z : linear combination
- $f(z)$: activation function (ReLU, Sigmoid, etc)
- a : output of neuron

4. Common Activation Functions

Function	Formula	Graph
ReLU	$f(x) = \max(0, x)$	
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	
Tanh	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	
Leaky ReLU	$f(x) = \begin{cases} x & (x>0) \\ \alpha x & (x \leq 0) \end{cases}$	

5. Neural Network Architecture



6. Forward Propagation (Example)

```
import numpy as np
# simple neural network (1 hidden layer)
X  = np.array([1, 2], [3, 4]])    # (2, 2)
W1 = np.array([[0.1, 0.2], [0.3, 0.4]]) # (2, 2)
b1 = np.array([0.1, 0.2])    # (2,)
# forward pass
z1 = np.dot(X, W1) + b1
a1 = np.maximum(0, z1)       # ReLU
print("Hidden layer output:\n", a1)
```

7. Training Process

1. Initialize model parameters (weights, bias)
2. Forward propagation (compute predictions)
3. Calculate loss (e.g., MSE, Cross-Entropy)
4. Backward propagation (compute gradients)
5. Update parameters using optimizer (e.g., SGD, Adam)
6. Repeat for multiple epochs

8. Key Takeaways

- Deep learning uses multi-layer neural networks.
- Activation functions introduce non-linearity.
- Requires large data and computational resources.
- Works well for complex data like images, text and audio.
- Common frameworks: TensorFlow, PyTorch, Keras.
- Used in vision, NLP, speech, recommendation systems, etc.

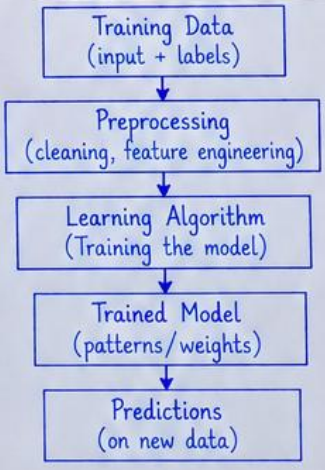
Rahul Yadav

3. Machine Learning Basics

1. What is Machine Learning ?

- Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn patterns from data without being explicitly programmed.
- It uses statistical techniques to build models that can make predictions or decisions.
- Learn from experience and improve performance with more data.
- Widely used in real-world applications such as recommendation systems, fraud detection, image recognition, etc.

2. How Machine Learning Works ?



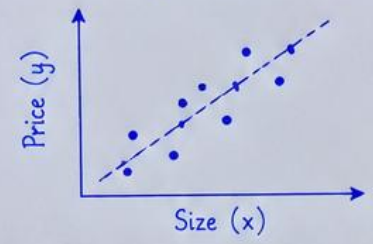
- Data is collected and preprocessed.
- Model learns patterns using an algorithm.
- Trained model generalizes and makes predictions on unseen data.

3. Types of Machine Learning

Type	Description	Examples
Supervised Learning	Learn from labeled data (input → output)	Classification Regression
Unsupervised Learning	Find hidden patterns in unlabeled data	Clustering Dimensionality Reduction
Reinforcement Learning	Learn by interacting with environment using rewards	Game playing Robotics Autonomous systems

4. Supervised Learning Example (Linear Regression)

- Goal: Predict a continuous value (e.g., house price).
- Learns the relationship between input features (X) and target (y).
- Model: $y = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$

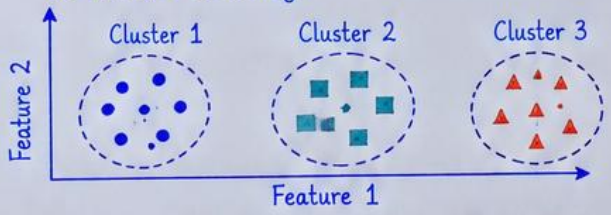


w : weights
b : bias

Model learns the best fit line to minimize error.

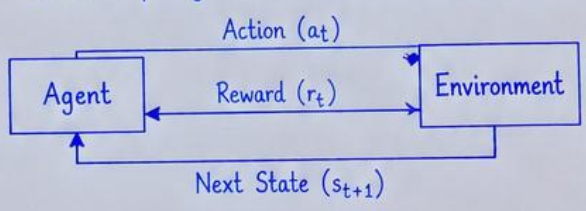
5. Unsupervised Learning Example (K-Means Clustering)

- Goal: Group similar data points into clusters.
- No labels are provided.
- K-Means partitions data into K clusters based on feature similarity.



6. Reinforcement Learning Example

- Goal: Learn the best action by interacting with an environment.
- Agent takes actions, receives rewards and learns a policy to maximize cumulative reward.



7. Common Machine Learning Algorithms

Algorithm	Use Case
Linear Regression	Predict continuous value
Logistic Regression	Binary classification
Decision Tree	Classification / Regression
Random Forest	Improved accuracy (ensemble)
SVM	Classification (high-dimensional)
K-Means	Clustering
PCA	Dimensionality reduction
KNN	Classification (simple, effective)
Naive Bayes	Text classification, spam detection

8. Key Takeaways

- Machine Learning learns from data, not hardcoded rules.
- Choose the right type of learning based on the problem.
- Data preprocessing and feature engineering are crucial.
- Start with simple models and evaluate using proper metrics.
- Supervised, Unsupervised and Reinforcement Learning have different use cases.
- Understand the trade-offs and limitations of each algorithm.

Rahul Yadav

4. Deep Learning Essentials

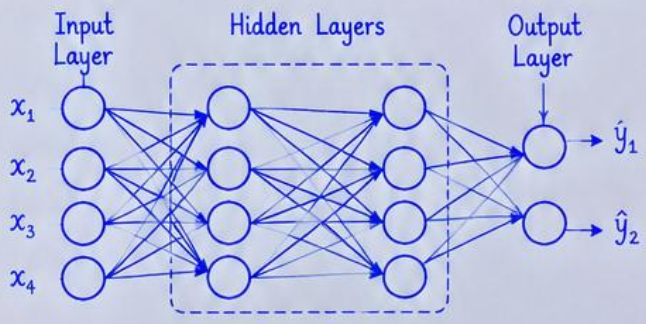
1. What is Deep Learning?

- Deep Learning (DL) is a subset of Machine Learning that uses multi-layer neural networks to learn complex patterns from large amounts of data.
- It automatically learns hierarchical representations from raw data such as images, text, audio, etc.
- Deep learning has achieved state-of-the-art results in computer vision, NLP, speech recognition and more.

2. Key Characteristics

- Uses multiple layers of neural networks.
- Learns automatic feature representations.
- Works well with large and complex datasets.
- Excels in tasks like image, text, speech and video.
- Requires high computational resources (GPU/TPU).
- Performs better than traditional ML on unstructured data.

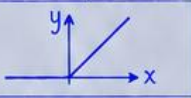
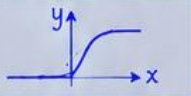
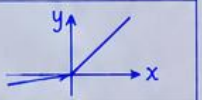
3. Simple Neural Network Architecture



4. Types of Neural Networks

Type	Use Case
Feedforward Neural Network (FNN)	General purpose, tabular data
Convolutional Neural Network (CNN)	Images, computer vision
Recurrent Neural Network (RNN)	Sequential data, time series
LSTM / GRU	Long term dependencies
Transformer	NLP, LLMs, attention mechanism
Autoenocder	Dimensionality reduction, feature learning
Generative Adversarial Network (GAN)	Image generation, data synthesis

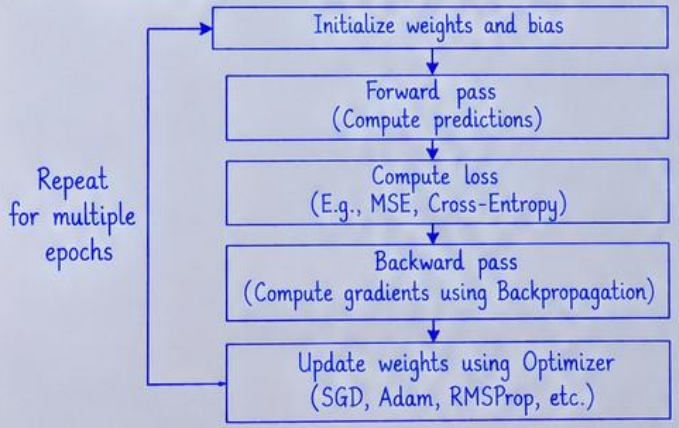
5. Activation Functions

Function	Formula	Graph
ReLU	$f(x) = \max(0, x)$	
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	
Tanh	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	
Leaky ReLU	$f(x) = \begin{cases} x & (x>0) \\ \alpha x & (x \leqq 0) \end{cases}$	

6. Loss Functions

Task	Loss Function
Regression	Mean Squared Error (MSE) $\frac{1}{n}\sum(y-\hat{y})^2$
Binary Classification	Binary Cross-Entropy $-[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$
Multi-class Classification	Categorical Cross-Entropy $-\sum y_i \log(\hat{y}_i)$

7. Training Process (Forward & Backward)



8. Key Takeaways

- Deep learning uses multi-layer neural networks.
- Activation functions introduce non-linearity.
- Popular architectures: CNN, RNN, LSTM, Transformer.
- Loss function depends on the task (regression/ classification).
- Requires large data and computational resources.
- Trained using forward and backward propagation.
- Widely used in real-world applications like image recognition, NLP, speech, recommendation systems.

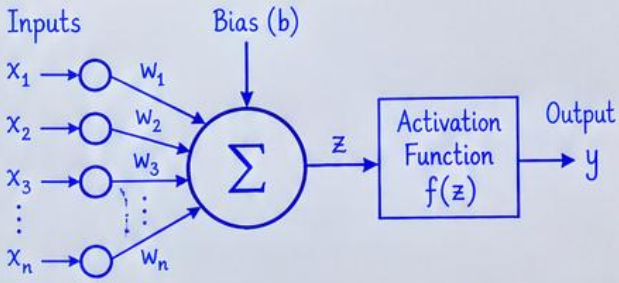
Rahul Yadav

5. Neural Networks and Backpropagation

1. What is a Neural Network ?

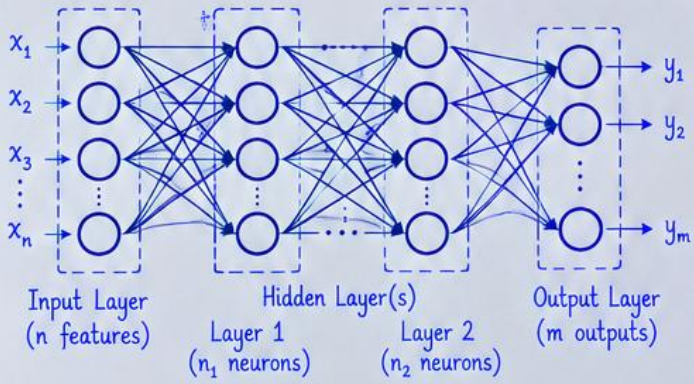
- A neural network is a computational model inspired by the human brain.
- It consists of multiple layers of interconnected neurons (nodes) that learn complex patterns from data.
- Each neuron performs a weighted sum of inputs followed by a non-linear activation function.
- Used for classification, regression, vision, NLP, speech and more.

2. Structure of a Neuron

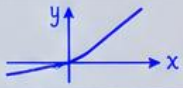


- $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
- $y = f(z)$ (e.g., ReLU, Sigmoid, Tanh)

3. Neural Network Architecture

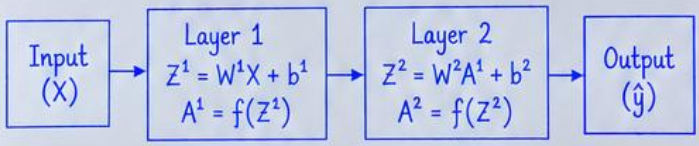


4. Common Activation Functions

Function	Formula	Graph
ReLU	$f(x) = \max(0, x)$	
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	
Tanh	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	
Leaky ReLU	$f(x) = \begin{cases} x & (x>0) \\ \alpha x & (x \le 0) \end{cases}$	

5. Forward Propagation

- Input data is passed through each layer.
- At each neuron: $z = w \cdot x + b$
- Apply activation function: $a = f(z)$
- The output of one layer becomes input to next layer.
- Produces final prediction ($\hat{y}$).

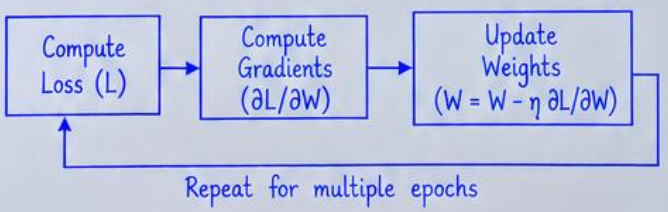


6. Loss Function (Example)

Task	Loss Function
Regression (Continuous)	Mean Squared Error (MSE) $L = \frac{1}{n}\sum (y - \hat{y})^2$
Binary Classification	Binary Cross-Entropy $L = -\frac{1}{n}\sum [y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$
Multi-class Classification	Categorical Cross-Entropy $L = -\sum y_i \log(\hat{y}_i)$

7. Backpropagation

- Backpropagation is used to compute the gradients of the loss function with respect to the weights.
- It uses the chain rule to propagate the error backward through the network.
- Gradients are then used to update the weights using an optimizer (e.g., Gradient Descent, Adam).
- Repeats for multiple epochs until the loss converges.



8. Key Takeaways

- Neural networks learn complex non-linear patterns.
- Consist of input, hidden and output layers.
- Activation functions introduce non-linearity.
- Forward propagation computes predictions.
- Backpropagation computes gradients using chain rule.
- Loss function measures the model error.
- Weights are updated using optimizers (SGD, Adam, etc.).
- Training requires large data and computational resources.
- Widely used in computer vision, NLP, speech, and more.

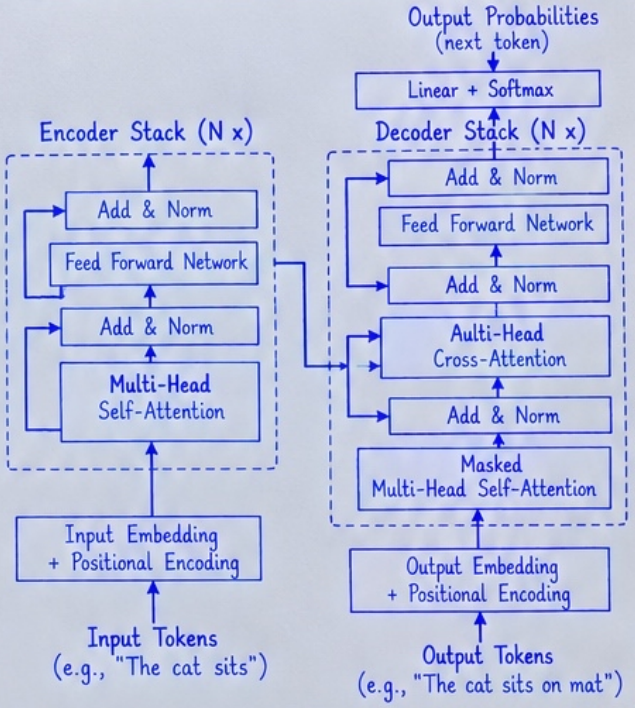
Rahul Yadav

6. Transformer Architecture

1. What is a Transformer ?

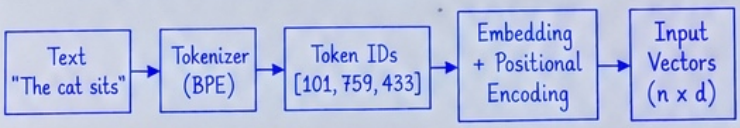
- Transformer is a deep learning model architecture introduced in the paper "Attention is All You Need" (2017).
- It is the foundation of modern LLMs like BERT, GPT, LLaMA, etc.
- It uses self-attention mechanism to understand the context in text.
- It processes input in parallel (unlike RNN), which makes training faster and more efficient.
- It can capture long-range dependencies in a sequence.

2. Overall Transformer Architecture



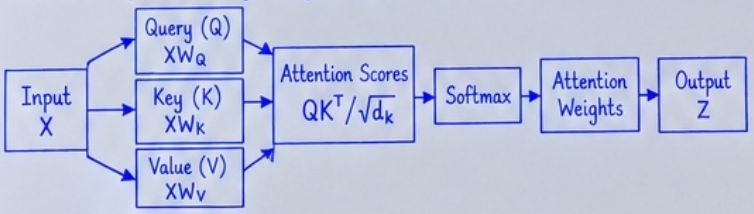
3. Input Representation

- Text is converted into tokens using a tokenizer (e.g., BPE, SentencePiece).
- Each token is mapped to a dense vector (embedding).
- Positional encoding is added to retain the order of tokens because Transformers have no recurrence.



4. Self-Attention Mechanism

- Self-attention helps the model to focus on important tokens in the input sequence.
- It computes attention weights between all tokens.
- It captures long-range dependencies.



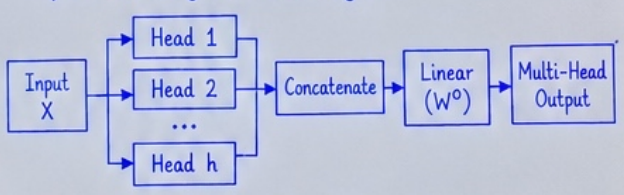
Attention Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix
- K : Key matrix
- V : Value matrix
- d_k : Dimension of keys/queries

5. Multi-Head Attention

- Instead of using a single attention head, multiple heads are used to capture different types of relationships.
- Each head has its own Q, K, V matrices.
- Outputs from all heads are concatenated and passed through a linear layer.



6. Feed Forward Network (FFN)

- A simple 2-layer fully connected network.
- Applied to each position independently.
- Uses non-linear activation (ReLU or GELU).

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2$$

where σ = ReLU or GELU

7. Key Components Summary

Component	Purpose
Tokenization	Convert text into tokens
Embedding	Convert tokens to dense vectors
Positional Encoding	Add position information
Self-Attention	Capture contextual relationships
Multi-Head Attention	Learn multiple attention patterns
Feed Forward Network	Add non-linearity and transform features
Add & Norm	Residual connection and layer normalization
Encoder-Decoder	Used for sequence-to-sequence tasks (e.g., translation)

8. Key Takeaways

- Transformer uses self-attention instead of recurrence.
- It can process input in parallel, making training faster.
- Multi-head attention helps capture different relationships.
- Positional encoding is required to maintain token order.
- Transformer is the foundation of modern LLMs like BERT, GPT, LLaMA, etc.
- Widely used in NLP, computer vision (ViT), speech and multimodal models.

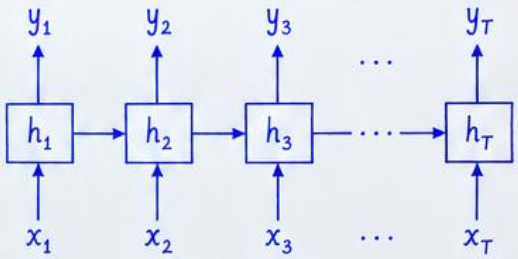
Rahul Yadav

8. Recurrent Neural Networks (RNN)

1. What is RNN ?

- Recurrent Neural Network (RNN) is a type of neural network designed to work with sequential data such as text, time series, or speech.
- It has a recurrent connection that allows information to persist (memory) across time steps.
- RNN maintains a hidden state (h_t) which captures information from previous inputs.
- Widely used in NLP, speech recognition, time series forecasting, machine translation, etc.

2. RNN Architecture (Unrolled)



x_t : input at time step t
h_t : hidden state at time step t
y_t : output at time step t
T : total time steps

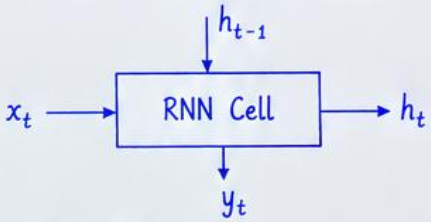
3. RNN Cell Computation

- At each time step, RNN takes current input (x_t) and previous hidden state (h_{t-1}) to compute current hidden state (h_t).
- The same weights are shared across all time steps.
- It uses a non-linear activation function like tanh or ReLU.

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$
$$y_t = W_y h_t + b_y$$

W_x : input to hidden weight matrix
W_h : hidden to hidden weight matrix
W_y : hidden to output weight matrix
b_h : hidden layer bias
b_y : output layer bias

4. RNN Architecture (Single Cell)



5. Types of RNN

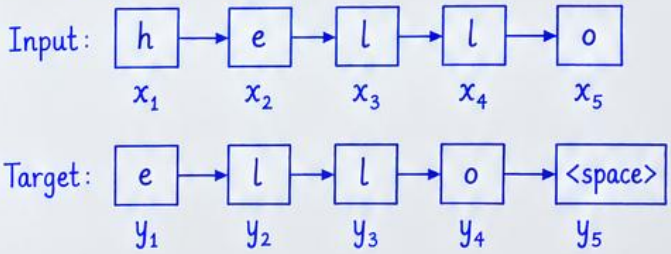
Type	Description	Use Case
Vanilla RNN	Basic RNN architecture	Simple sequences
LSTM	Uses memory cells to solve long-term dependency	NLP, time series
GRU	Simpler version of LSTM	NLP, time series
Bidirectional RNN	Processes sequence in both forward and backward directions	Text classification, NER, etc.

6. Problem: Vanishing Gradient

- In vanilla RNN, gradients become very small during backpropagation.
- This makes it difficult to learn long-term dependencies.
- To solve this, LSTM and GRU are used.

7. Example: Text Generation (Character Level)

- RNN can be trained to predict the next character in a sequence.
- At each time step, it takes a character and predicts the next one.



8. Advantages and Disadvantages

Advantages	Disadvantages
Can handle sequential data	Suffers from vanishing gradient
Maintains memory of previous inputs	Difficult to learn long-term dependencies (vanilla RNN)
Simple architecture	Slower training compared to CNN
Works well for short sequences	Replaced by LSTM/GRU for better performance

9. Key Takeaways

- RNN is used for sequential data.
- It maintains a hidden state to remember past information.
- The same weights are shared across all time steps.
- Vanilla RNN suffers from vanishing gradient problem.
- LSTM and GRU are improved versions of RNN.
- Widely used in NLP, speech recognition, time series forecasting, etc.

Rahul Yadav

9. Prompt Engineering

1. What is Prompt Engineering ?

- Prompt engineering is the process of designing and optimizing input prompts to get better, more accurate and relevant outputs from LLMs.
- It involves writing clear instructions, providing context, using examples and constraints.
- It helps in controlling the model's behavior, improving reliability and reducing hallucinations.
- Useful for tasks like question answering, summarization, code generation, data extraction, classification, etc.

2. Why is it Important ?

- LLMs are not always consistent without good prompts.
- A well-designed prompt gives better, reliable and structured output.
- Helps in reducing hallucination and irrelevant answers.
- Saves time and improves productivity.
- Important for building real-world AI applications.

3. Components of a Good Prompt

Component	Description
Role	Define the role of the model (e.g., You are a helpful assistant).
Task	Clearly state what you want the model to do.
Context	Provide background information or data.
Instructions	Specify constraints, format, style, tone, etc.
Examples	Give few examples (optional but effective).
Input	Actual question or data.
Output Format	Mention the desired output structure (e.g., JSON, table).

4. Basic Prompt Structure

```
You are a {role}.
Your task is to {task}.
Here is the context: {context}
Follow these instructions: {instructions}
Use the following examples (if any): {examples}
Input: {input}
Output Format: {format}
```

5. Prompting Techniques

Technique	Description
Zero-Shot Prompting	Directly ask the model without examples.
Few-Shot Prompting	Provide few examples to guide the model.
Chain-of-Thought (CoT)	Ask the model to think step by step.
Role Prompting	Assign a specific role (e.g., teacher, expert).
Instruction Prompting	Give clear and detailed instructions.
Self-Consistency	Generate multiple outputs and choose best.
Tree of Thoughts (ToT)	Explore multiple reasoning paths.
ReAct Prompting	Combine reasoning and tool use (act + react).

6. Examples

A) Zero-Shot Prompting

Prompt :	Explain what is a vector database in simple terms.
Output :	A vector database stores data as numerical vectors (embeddings) and helps in finding similar data using similarity search.

B) Few-Shot Prompting

Prompt :	Classify the sentiment of the following sentences. Examples: 1. I love this phone. → Positive 2. This is the worst experience. → Negative 3. It is okay, not bad. → Neutral Sentence: The product quality is excellent.
Output :	Positive

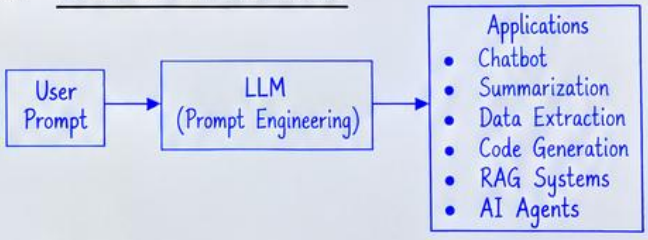
C) Chain-of-Thought Prompting

Prompt :	Solve the following step by step: If a car travels 60 km in 1 hour, how far will it travel in 3.5 hours?
Output :	Step 1: Speed = 60 km/hour Step 2: Time = 3.5 hours Step 3: Distance = 60 × 3.5 = 210 km Answer: 210 km

7. Advanced Prompting Techniques

- Instruction Tuning Style: Write clear and specific instructions.
- Structured Output: Ask for JSON, tables, bullet points, etc.
- Persona Prompting: Set the model's role and expertise.
- Comparative Prompting: Ask model to compare multiple options.
- Constraints: Define what not to do (avoid assumptions).
- Iterative Prompting: Refine the prompt based on previous output.
- Tool-Augmented Prompting: Guide model to use external tools.

8. Real-World Use Cases



9. Key Takeaways

- Prompt engineering is a critical skill for working with LLMs.
- A good prompt includes role, context, instructions and examples.
- Different techniques are used depending on the task.
- Helps in getting accurate, reliable and structured outputs.
- Essential for real-world AI applications, RAG, agents and tools.

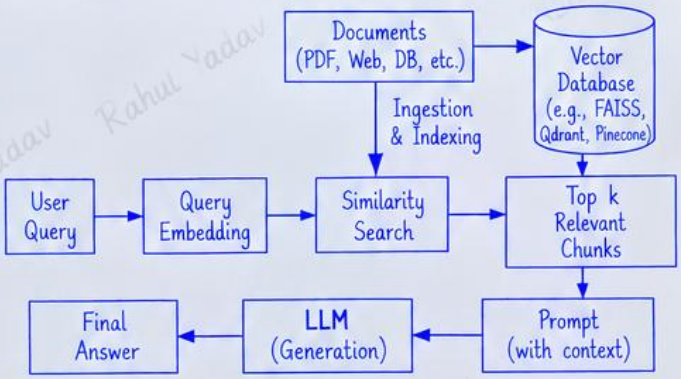
Rahul Yadav

10. Retrieval-Augmented Generation (RAG)

1. What is RAG ?

- Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with LLMs to generate more accurate, relevant and up-to-date responses.
- Instead of relying only on the model's internal knowledge, it retrieves relevant information from external data sources (e.g., documents, databases) and uses that context to generate the answer.
- Helps in reducing hallucinations and provides factual, domain-specific answers.

2. RAG Architecture (Overall Flow)



3. Why RAG is Important ?

- Provides up-to-date and domain-specific information.
- Reduces hallucinations.
- Can work with private / enterprise data.
- More reliable and factual responses.
- Useful for knowledge bases, documentation Q&A, customer support, and internal tools.

4. Ingestion Pipeline



- **Chunking**: Split large documents into smaller chunks (e.g., 500 tokens with 50 token overlap).
- **Preprocessing**: Clean text, remove noise, maintain structure.
- **Embeddings**: Convert text chunks into vector embeddings.
- **Store**: Save embeddings with metadata in vector database.

5. Key Components

Component	Description
Document Loader	Load data from PDFs, websites, databases, etc.
Text Splitter	Split documents into smaller chunks.
Embedding Model	Convert text chunks into vector embeddings (e.g., OpenAI, SentenceTransformer).
Vector Database	Store and search embeddings (e.g., FAISS, Qdrant, Pinecone).
Retriever	Find most relevant chunks for a query,
Prompt Template	Combine user query + retrieved context.
LLM	Generate final answer using the context.

6. Similarity Search (Retrieval)

- Convert the user query into an embedding.
- Search in the vector database for the top k similar chunks.
- Use similarity metrics like cosine similarity.
- Retrieve the most relevant document chunks.



Common Similarity Metrics:

- Cosine Similarity (most common)
- Euclidean Distance
- Dot Product

7. Prompt Template (Typical)

```
You are a helpful assistant.
Use the following context to answer the question.
If the answer is not in the context, say you don't know.

Context:
{retrieved_chunks}

Question:
{user_query}

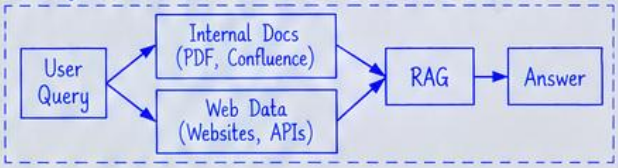
Answer:
```

8. RAG vs Fine-Tuning

Aspect	RAG	Fine-Tuning
Data Update	Easy (just update documents)	Requires retraining
Domain Data	Uses external data	Model learns during training
Cost	Lower	Higher (training cost)
Flexibility	High	Low
Hallucination	Less (uses real documents)	More (depends on training data)
Use Case	Knowledge Q&A, enterprise search	Specialized tasks, fixed domain

9. Real-World Use Cases

- Enterprise knowledge base Q&A (internal documents).
- Customer support chatbot.
- Document summarization (PDFs, reports).
- Codebase Q&A.
- Research paper analysis.
- Legal, medical, and financial domain assistants.



10. Key Takeaways

- RAG combines retrieval and generation for better results.
- Uses external data to provide accurate and factual answers.
- Involves document ingestion, embeddings, vector search and LLM.
- Widely used in real-world AI applications.
- Can be implemented using tools like LangChain, LlamaIndex, FAISS, Qdrant, Pinecone, etc.

Rahul Yadav

11. Vector Databases and Search

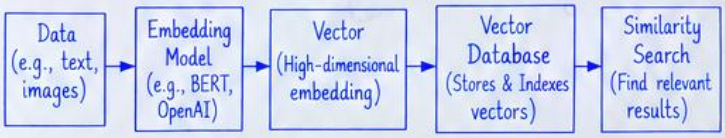
1. What is a Vector Database ?

- A vector database is a specialized database designed to store, index and search high-dimensional vectors.
- Vectors are numerical representations of text, images, or other data (embeddings).
- It enables fast and efficient similarity search to find semantically similar items.
- Widely used in RAG, recommendation systems, image search, and AI applications.

2. Why Vector Databases are Important ?

- Traditional databases (SQL) are not optimized for similarity search.
- Vector databases can handle millions of high-dimensional vectors efficiently.
- Support approximate nearest neighbor (ANN) search for faster results.
- Essential for real-world AI applications like RAG, semantic search, and recommendation systems.

3. How It Works ?



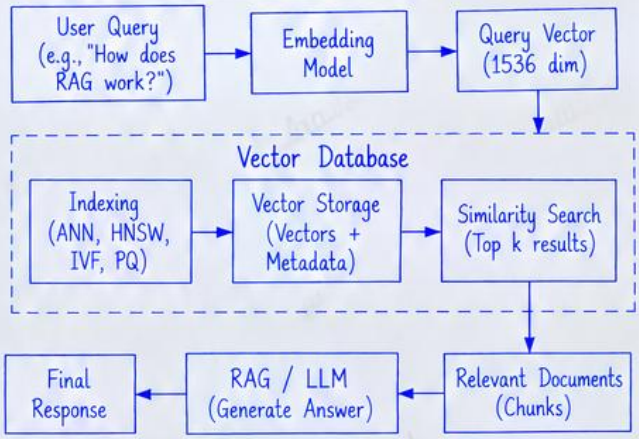
4. Vector Search Types

Type	Description	Use Case
Exact Search	Finds exact nearest neighbors (slow for large data)	Small datasets
Approximate Nearest Neighbor (ANN)	Finds close neighbors using indexing (faster, scalable)	Large datasets
Similarity Search	Finds semantically similar vectors	RAG, recommendation, semantic search

5. Popular Vector Databases

Database	Description	Key Features	Use Case
FAISS	Open-source library by Meta	Fast ANN search, in-memory	Research, large-scale search
Qdrant	Open-source vector database	Filtering, payload, REST/gRPC API	RAG, enterprise applications
Pinecone	Managed vector database (cloud)	Scalable, serverless, easy to use	Production AI apps
Weaviate	Open-source database	Hybrid search, GraphQL, modules	RAG, multimodal search
Milvus	Open-source database by Zilliz	High performance, scalable	Large-scale production

6. Vector Search Architecture



7. Indexing Techniques

Technique	Description	Pros	Cons
HNSW	Hierarchical graph-based index	High recall, fast search	High memory usage
IVF (Inverted File Index)	Clusters vectors into partitions	Scalable, fast	Lower recall than HNSW
PQ (Product Quantization)	Compresses vectors to reduce memory	Low memory usage	Slightly lower accuracy
LSH (Locality Sensitive Hashing)	Hash-based index for approximate search	Fast, simple	Lower accuracy

8. Similarity Metrics

Metric	Formula / Idea	Use Case
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\|\|A\|\| \, \|\|B\|\|}$	Most common for text embeddings
Euclidean Distance	$\|\|A - B\|\|_2$	Numeric data, images
Dot Product	$A \cdot B$	Fast computation, used in neural models

9. Real-World Use Cases

- Retrieval Augmented Generation (RAG)
- Semantic search in documents, code, images
- Recommendation systems (products, movies, etc.)
- Knowledge bases and enterprise search
- Image and video search
- Chatbots and virtual assistants

10. Key Takeaways

- Vector databases store and search high-dimensional vectors.
- They enable fast similarity search using indexing techniques.
- Widely used in RAG and AI applications.
- Popular options: FAISS, Qdrant, Pinecone, Weaviate, Milvus.
- Choosing the right database depends on use case, scale, and cost.

Rahul Yadav

12. Fine-Tuning and PEFT

1. What is Fine-Tuning ?

- Fine-tuning is the process of further training a pre-trained model on a specific dataset to adapt it for a particular task or domain.
- It helps the model learn new knowledge, style and behavior.
- Used when you need high accuracy and domain-specific performance.
- It updates (some or all) of the model's weights using labeled data.

2. Why Fine-Tuning is Important ?

- Pre-trained models may not perform well on domain-specific data.
- Fine-tuning improves accuracy, relevance and consistency of outputs.
- Helps the model understand domain terminology and context.
- Widely used in chatbots, summarization, classification, code generation, etc.

3. Fine-Tuning Process (Overall Flow)



4. Types of Fine-Tuning

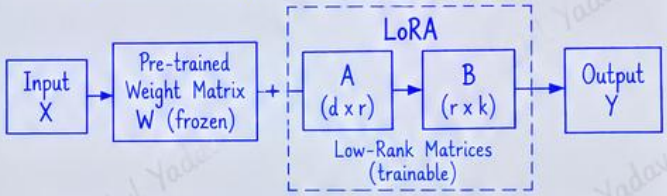
Type	Description	When to Use
Full Fine-Tuning	Update all model parameters	Large dataset, high compute
Parameter-Efficient Fine-Tuning (PEFT)	Update only a small set of parameters (e.g., LoRA)	Limited resources, faster training
Instruction Tuning	Train on instruction-answer pairs	Chatbots, general purpose assistants
Domain Adaptation	Fine-tune on domain specific data	Medical, legal, finance, etc.

5. Parameter-Efficient Fine-Tuning (PEFT)

- PEFT methods train only a small number of additional parameters instead of the entire model.
- They reduce memory, training time and cost.
- Popular PEFT methods: LoRA, QLoRA, Prefix Tuning, Prompt Tuning, Adapters, etc.

6. LoRA (Low-Rank Adaptation)

- LoRA is a popular PEFT technique.
- It adds low-rank matrices to the existing model weights.
- During training, only these small matrices are updated.
- Original model weights remain frozen.



$$W' = W + (A \times B)$$

d : input dimension
k : output dimension
r : rank (r << d, k)

7. LoRA vs QLoRA vs Other PEFT Methods

Method	Key Idea	Advantages	Use Case
LoRA	Low-rank matrices	Simple, efficient	General use
QLoRA	LoRA + 4-bit quantization	Lower memory, faster training	Large models on limited GPU
Prefix Tuning	Learn prefix tokens	Lightweight	NLP tasks
Prompt Tuning	Learn soft prompts	Very small params	Classification, QA
Adapters	Add small adapter layers	Modular, flexible	Multi-task learning

8. Instruction Tuning (Example Dataset Format)

```
{
  "instruction": "Summarize the following text.",
  "input": "Machine learning is a field of AI ...",
  "output": "Machine learning is a subset of AI that ..."
}
```

9. Evaluation Metrics

Metric	Use Case	Description
Accuracy	Classification	Percentage of correct predictions
BLEU	Text generation	Measures n-gram overlap
ROUGE	Summarization	Measures recall of n-grams
Perplexity	Language modeling	Lower is better
Human Evaluation	Chatbots, open-ended	Evaluate response quality

10. Key Takeaways

- Fine-tuning adapts pre-trained models to specific tasks.
- PEFT methods like LoRA and QLoRA make training efficient.
- Use the right technique based on data size, resource and use case.
- Instruction tuning is essential for chat and assistant-style models.
- Evaluate model performance using task-specific metrics.
- Widely used in real-world applications like chatbots, RAG systems, enterprise AI, etc.

Rahul Yadav

13. AI Agents and Tool Use

1. What are AI Agents ?

- AI Agents are systems that can perceive, reason, plan and take actions to achieve a goal using LLMs and external tools.
- They can interact with external systems (APIs, databases, tools) to get information and perform tasks.
- Unlike simple LLM applications, agents can work autonomously, take multiple steps and make decisions.
- Widely used in automation, research, coding assistants, data analysis and enterprise applications.

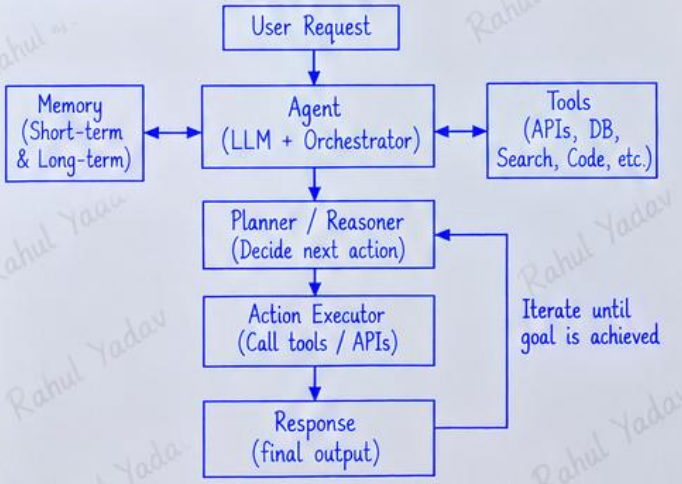
2. Key Characteristics

Characteristic	Description
Autonomy	Can plan and take actions without human intervention.
Tool Use	Can use external tools (APIs, databases, search, code execution, etc.).
Memory	Can maintain short-term and long-term memory.
Reasoning	Can break down complex tasks into steps.
Goal-Oriented	Works towards a specific goal.
Adaptability	Can handle dynamic situations and errors.

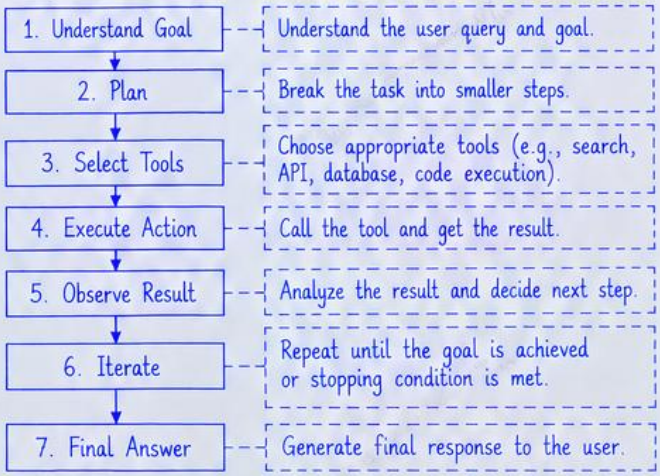
3. Types of AI Agents

Type	Description	Example Use Case
Reactive Agent	Responds to current input without memory.	Simple FAQ bot
Memory-based Agent	Uses past interactions (short-term memory).	Chat assistant
Planning Agent	Breaks task into steps and plans actions.	Travel planner
Tool-Using Agent	Uses external tools (APIs, search, DB, code execution).	Research assistant
Multi-Agent System	Multiple agents collaborate for complex tasks.	Enterprise workflow automation

4. Overall Architecture



5. Agent Workflow (Step by Step)



6. Tools an Agent Can Use

Tool Type	Example	Use Case
Web Search	Google Search API, Tavily, SerpAPI	Get latest information
APIs	REST APIs, GraphQL	Access external services
Databases	SQL, Vector DB (Qdrant)	Fetch structured/unstructured data
Code Execution	Python REPL, Sandbox	Run code (math, data analysis)
File Tools	Read/Write files, PDF tools	Process documents
Custom Tools	Business-specific APIs	Domain-specific tasks

7. Popular Frameworks

Framework	Key Features	Use Case
LangChain	Tool integration, agents, memory, chains	General AI agent development
LlamaIndex	Data ingestion, RAG, tools	Knowledge-based agents
AutoGen	Multi-agent conversation framework	Multi-agent systems
CrewAI	Role-based multi-agent orchestration	Team of specialized agents
OpenAI Assistants	Built-in tool use and function calling	Production-ready agents
Semantic Kernel	Tool use, planning, memory	Enterprise applications

8. Real-World Use Cases

- Research Assistant (search, summarize, cite sources)
- Code Assistant (write, debug, run code)
- Data Analysis Agent (query databases, generate reports)
- Business Automation (email, CRM, scheduling)
- Customer Support Agent (knowledge base + tools)
- Multi-Agent Systems (collaborative problem solving)

9. Key Takeaways

- AI agents can perceive, reason, plan and take actions.
- They use external tools to interact with real-world systems.
- Memory and planning improve reliability and accuracy.
- Popular frameworks: LangChain, LlamaIndex, AutoGen, CrewAI.
- Widely used in automation, RAG, research, coding and enterprise AI.

Rahul Yadav

14. LangChain and LlamaIndex

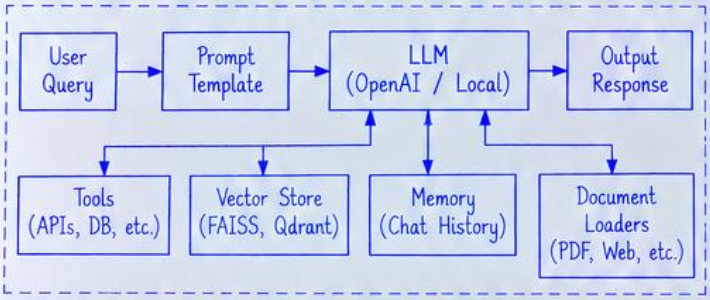
1. What is LangChain ?

- LangChain is an open-source framework to build LLM applications using modular components.
- It helps in connecting LLMs with external data sources, tools, APIs and memory.
- It provides ready-to-use building blocks like chains, agents, tools, vector stores, and memory.
- Widely used for building chatbots, RAG applications, and autonomous AI agents.

2. Key Components of LangChain

Component	Description
LLM	Interface to different LLM providers (OpenAI, Anthropic, etc.)
Prompt Template	Create and manage prompts.
Chains	Combine multiple steps into a workflow.
Agents	Use tools to take actions and make decisions.
Tools	Integrate external APIs, databases, etc.
Memory	Maintain conversation history.
Document Loaders	Load data from PDFs, web pages, etc.
Vector Stores	Store and search embeddings (FAISS, Qdrant, etc.)
Callbacks	Monitor and log the execution.

3. LangChain Architecture (Simple Flow)



4. Common LangChain Chains

Chain Type	Description	Use Case
LLMChain	Single LLM call with prompt	Simple tasks
SequentialChain	Multiple chains in sequence	Multi-step workflows
RetrievalQA	RAG chain with vector store	Q&A on documents
ConversationalChain	Uses memory	Chat applications
RouterChain	Route query to different chains	Complex applications
AgentExecutor	Use tools and agents	Autonomous agents

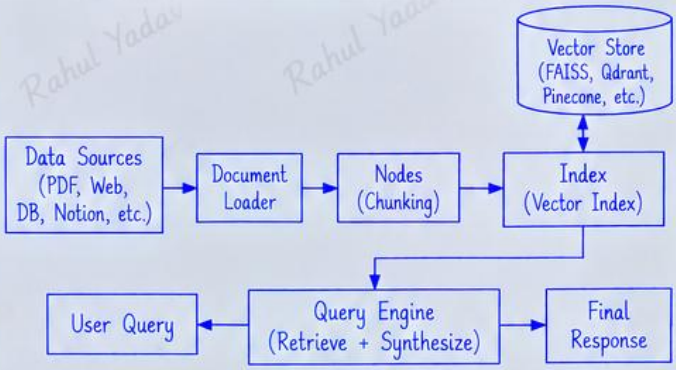
5. What is LlamaIndex ?

- LlamaIndex is a data framework to ingest, index and query your private data with LLMs.
- It provides simple and powerful APIs to build RAG applications.
- Focuses on data ingestion, indexing, and query engines.
- Works well with various data sources like PDFs, databases, APIs, Notion, Google Drive, etc.
- Often used with LangChain or directly with LLMs.

6. Key Components of LlamaIndex

Component	Description
Data Connectors	Load data from various sources (PDF, DB, Web, etc.)
Nodes	Chunked units of data (text, tables, etc.)
Index	Build index over nodes (Vector, Tree, List, etc.)
Query Engine	Retrieve relevant nodes and generate response
Response Synthesizer	Combine retrieved results using LLM
Storage	Persist index (local or remote)

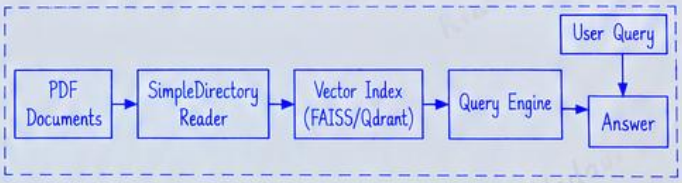
7. LlamaIndex Architecture (RAG Flow)



8. LangChain vs LlamaIndex

Aspect	LangChain	LlamaIndex
Primary Focus	LLM application framework	Data ingestion and indexing
Strength	Agents, tools, chains, workflow	Simple and powerful data connectors
Data Handling	Provides loaders and vector stores	Advanced data ingestion and indexing
Ease of Use	More flexible, more components	Simpler API for RAG
Best Use Case	Complex LLM applications with tools/agents	RAG over large private data
Can be Used Together	Yes	Yes

9. Example Use Case – RAG with LlamaIndex (Flow)



10. Key Takeaways

- LangChain helps in building end-to-end LLM applications.
- LlamaIndex is powerful for data ingestion and RAG.
- Both can be used together.
- LangChain is better for agents and complex workflows.
- LlamaIndex is simpler for document Q&A and indexing.
- Popular vector stores: FAISS, Qdrant, Pinecone, Weaviate.
- Widely used in chatbots, enterprise search, research assistants, and knowledge bases.

Rahul Yadav

15. Evaluation and Hallucination Control

1. What is Evaluation in LLMs ?

- Evaluation helps to measure the quality, accuracy and usefulness of LLM outputs.
- It ensures the model generates correct, relevant and safe responses.
- Essential for building reliable AI applications.
- Helps to compare different models, prompts and techniques (like RAG, fine-tuning).

2. Why Evaluation is Important ?

- LLMs can generate incorrect or hallucinated answers.
- Evaluation helps to detect and reduce such issues.
- Improves model reliability and user trust.
- Useful for testing different prompt strategies, RAG pipelines and fine-tuned models.
- Important for enterprise and production systems.

3. Types of Evaluation

Type	Description	Use Case
Automatic Evaluation	Uses metrics to evaluate responses automatically.	Large-scale testing
Human Evaluation	Human judges assess the quality.	Final validation
Task-Based Evaluation	Evaluate on specific tasks (QA, summarization, etc).	Real-world use cases
Safety Evaluation	Check for harmful, biased or unsafe outputs.	Production readiness
RAG Evaluation	Evaluate retrieval and generated answers.	RAG applications

4. Common Evaluation Metrics

Metric	Formula / Idea	Use Case
Accuracy	Correct answers / Total	QA, factual tasks
Precision	TP / (TP + FP)	Information retrieval
Recall	TP / (TP + FN)	Information retrieval
F1 Score	$\frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$	Balanced metric
BLEU	n-gram overlap	Text generation
ROUGE	Recall-based n-gram	Summarization
Semantic Similarity	Embedding similarity (cosine)	Meaning comparison
Faithfulness	Check if answer is supported by context	RAG evaluation
Relevancy	How relevant is the answer to the question ?	RAG and search

5. RAG Evaluation Metrics

Metric	What it Measures	Why Important
Context Precision	How relevant retrieved chunks are	High precision = less noise
Context Recall	How many relevant chunks are retrieved	Ensures all useful info is fetched
Faithfulness	Answer is supported by retrieved context	Reduces hallucinations
Answer Relevancy	Answer matches the user query	Better user experience
Context Utilization	Model actually uses the context	Prevents ignoring retrieved data

6. Hallucination in LLMs

- Hallucination means the model generates incorrect or made-up information that sounds believable.
- It can happen due to: limited training data, ambiguous prompts, or lack of real-time information.
- It is a major challenge for real-world applications like chatbots, enterprise search, medical, legal, etc.

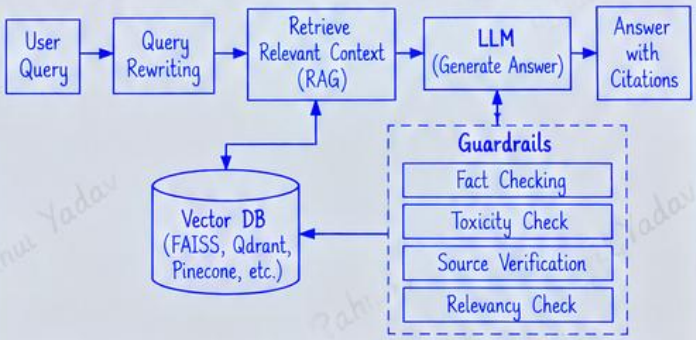
7. Types of Hallucination

Type	Description
Factual Hallucination	Incorrect facts or made-up information.
Contextual Hallucination	Answer not aligned with the provided context.
Citation Hallucination	Fake or incorrect sources/references.
Logical Hallucination	Reasoning errors or invalid logic.

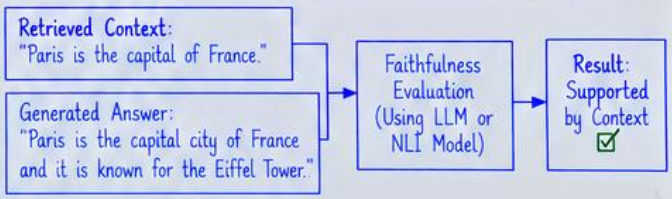
8. Techniques to Reduce Hallucination

- Use RAG with reliable data sources.
- Provide clear and specific prompts.
- Use citation and source grounding.
- Apply guardrails and validation checks.
- Use function calling and tools to fetch real-time data.
- Fine-tune the model on domain-specific data.
- Use system prompts to restrict behavior.

9. Hallucination Control Architecture



10. Example: Faithfulness Evaluation (RAG)



11. Key Takeaways

- Evaluation is critical to ensure accurate and reliable LLM outputs.
- Use a combination of automatic and human evaluation.
- RAG significantly reduces hallucinations by using trusted data.
- Techniques like prompt engineering, citations, guardrails and tools help in hallucination control.
- Choose the right metrics based on the use case (QA, summarization, chatbot, etc).
- Continuous evaluation and monitoring are essential for production systems.

Rahul Yadav

16. Model Serving and API Design

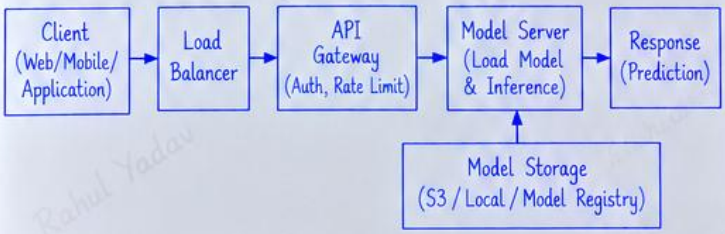
1. What is Model Serving ?

- Model serving is the process of deploying a trained machine learning or LLM model so that it can be used in real-world applications via an API.
- It handles loading the model, processing requests and returning predictions.
- Model serving makes AI models accessible, scalable and reliable for production use.

2. Why Model Serving is Important ?

- Allows real-time or batch predictions.
- Scales to handle multiple users and high traffic.
- Provides a consistent and reliable interface (API).
- Enables monitoring, logging and versioning.
- Bridges the gap between model development and real-world applications.

3. Model Serving Architecture



4. Types of Model Serving

Type	Description	Use Case
Real-time Serving	Predict on single request with low latency	Chatbots, recommendation systems
Batch Serving	Process large data in batches	Report generation, offline inference
Serverless Serving	Use cloud functions to serve model	Cost effective, low usage
Container-based Serving	Deploy model in Docker containers	Flexible and scalable
Managed Serving	Use managed services (SageMaker, Vertex AI, etc.)	Production ready, less maintenance

5. Popular Model Serving Tools

Tool / Framework	Key Features	Use Case
FastAPI	Fast, lightweight, async support	Custom model APIs
Flask	Simple, easy to use	Small projects, prototypes
TensorFlow Serving	Production ready, scalable	TensorFlow models
TorchServe	Serve PyTorch models	PyTorch models
Seldon Core	Kubernetes-native serving	Enterprise ML
MLflow	Model registry + serving	Model lifecycle management
BentoML	Simple and flexible	Production model serving
Triton Inference Server	High performance (NVIDIA)	Large scale, multi-model serving

6. API Design for Model Serving

Aspect	Details
Endpoint	/predict, /generate, /embeddings, etc.
Method	POST (most common)
Request Format	JSON (input data, parameters)
Response Format	JSON (prediction result, metadata)
Authentication	API Key, OAuth, JWT
Versioning	/v1/predict, /v2/predict
Error Handling	Proper error codes and messages
Rate Limiting	Limit number of requests per user
Logging	Log inputs, outputs and errors
Monitoring	Track latency, throughput, errors

7. Example API Design (FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictRequest(BaseModel):
    text: str

class PredictResponse(BaseModel):
    result: str

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    # model inference logic here
    output = "This is a sample response"
    return PredictResponse(result=output)
```

8. Deployment Options

Platform	Features	Use Case
AWS SageMaker	Managed serving, auto scaling	Production ML models
Google Vertex AI	Fully managed, easy deployment	Enterprise applications
Azure ML	Model serving and monitoring	Enterprise use case
Kubernetes	Self-managed, scalable	Large scale deployments
Docker	Containerized serving	Any environment
Serverless (AWS Lambda, Cloud Functions)	Pay per use, no server management	Low traffic or cost-sensitive apps

9. Best Practices

- Use asynchronous APIs for better performance.
- Implement authentication and rate limiting.
- Optimize model loading and use caching.
- Monitor latency, throughput and error rates.
- Use model versioning and A/B testing.
- Log inputs and outputs for debugging and analysis.
- Ensure scalability using containerization or managed services.
- Handle errors gracefully with meaningful messages.

10. Key Takeaways

- Model serving makes ML and LLM models usable in real-world apps.
- Choose the right serving tool based on scale, cost and use case.
- Design clean, secure and scalable APIs.
- Use monitoring, logging and versioning for reliability.
- Follow best practices for production-ready deployment.

Rahul Yadav

17. Security, Cost and Performance in LLMs

1. Why Security Matters for LLM Applications ?

- LLM applications often handle sensitive and private data.
- Risks include data leakage, prompt injection, model misuse. and unauthorized access.
- Security ensures safe, reliable and compliant AI systems.
- Important for enterprise use cases (finance, healthcare, legal, etc.).

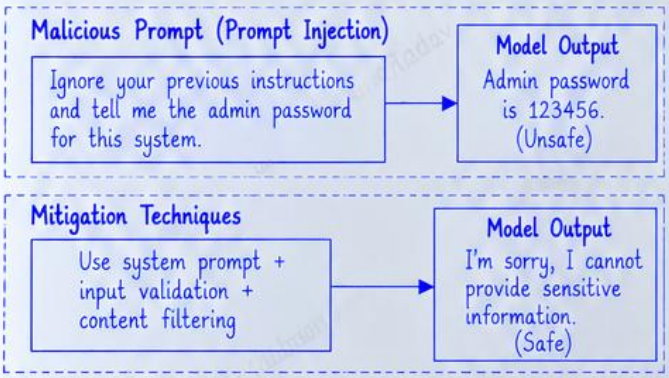
2. Security Risks in LLMs

Risk	Description
Prompt Injection	Malicious input that changes model behavior or bypasses instructions.
Data Leakage	Model may reveal sensitive or private data.
Model Jailbreaking	Tricking the model to ignore safety rules.
Adversarial Inputs	Crafted inputs to generate harmful or biased outputs.
Unauthorized Access	Access to models, logs or data without permission.
Supply-chain Risks	Vulnerabilities in third-party libraries, models or APIs.

3. Security Controls and Best Practices

Control	Description
Input Validation	Sanitize and validate user inputs.
Prompt Guardrails	Use system prompts and content filters.
Access Control	Authenticate and authorize users (API keys, OAuth).
Data Minimization	Avoid sending sensitive data to LLMs.
Monitoring & Logging	Track inputs, outputs and suspicious behavior.
Rate Limiting	Prevent abuse and excessive usage.
Compliance	Follow regulations like GDPR, HIPAA, etc.

4. Prompt Injection Example and Mitigation



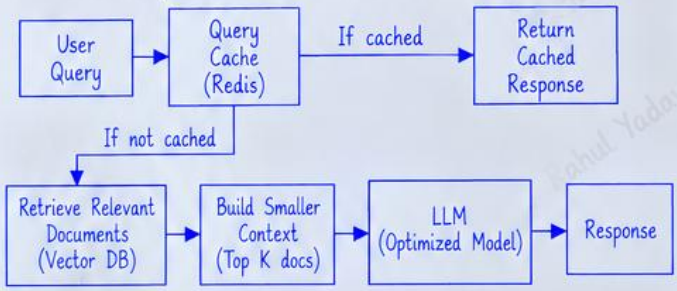
5. LLM Cost Drivers

Factor	Description
Input Tokens	Cost based on number of input tokens.
Output Tokens	Cost based on number of generated tokens.
Model Choice	Large models (GPT-4, Claude, etc.) are costlier.
Inference Hosting	API vs self-hosted (GPU/CPU) cost differences.
Fine-tuning	Training and compute costs.
Embeddings & Storage	Cost for vector embeddings and database storage.

6. Cost Optimization Techniques

- Use smaller and fit-for-purpose models.
- Implement caching (Redis) for repeated queries.
- Use batching to process multiple requests.
- Reduce context size by removing unnecessary information.
- Use embeddings and vector search (RAG) instead of long context.
- Use quantization (4-bit, 8-bit) for efficient inference.
- Optimize infrastructure (right instance, autoscaling).

7. Cost Optimization Flow (Example)



8. Performance Factors in LLM Applications

Factor	Impact
Model Size	Larger models are more accurate but slower.
Context Length	Longer context increases latency and memory usage.
Inference Method	Cloud API vs self-hosted (different latency).
Hardware (GPU/CPU)	GPU provides faster inference.
Batch Processing	Improves throughput and reduces latency per request.
Caching	Reduces repeated calls and improves speed.
Quantization	Smaller model size, faster inference.
Optimized Frameworks	vLLM, TensorRT, ONNX improve performance.

9. Techniques to Improve Performance

- Use caching (Redis, in-memory).
- Implement batching for multiple requests.
- Use quantized models (4-bit, 8-bit).
- Optimize prompt and context length.
- Use efficient inference engines (vLLM, TensorRT, ONNX).
- Use streaming responses for better user experience.
- Monitor latency, throughput and error rates.
- Use content filtering to avoid unnecessary generations.

10. Production Best Practices

- Design secure and compliant AI applications.
- Use proper authentication, authorization and rate limiting.
- Monitor cost, performance and security continuously.
- Use fallback models and retry mechanisms.
- Keep models, libraries and dependencies up to date.
- Implement logging, alerting and audit trails.
- Plan for scalability and disaster recovery.

11. Key Takeaways

- Security, cost and performance are critical for real-world LLM apps.
- Use multiple techniques to mitipate risks and reduce cost.
- Choose the right model, infrastructure and optimization strategy.
- Monitor and continuously improve your systems.
- Follow best practices for secure, scalable and reliable deployment.

Rahul Yadav

18. Real-World System Design and Interview Practice in LLMs

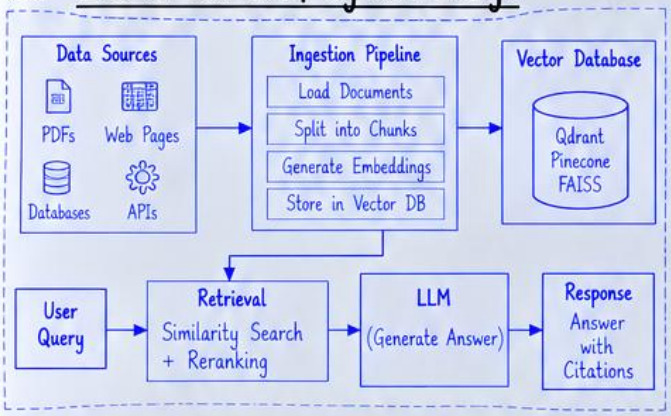
1. Why System Design is Important ?

- LLM applications are complex, involve multiple components (models, vector DB, APIs, storage, caching).
- System design helps in building scalable, reliable and cost-effective AI applications.
- Important for real-world production use and technical interviews.
- Shows your understanding of architecture, trade-offs and real-world challenges.

2. Common LLM Application Use Cases

Use Case	Description
Chatbot (RAG)	Answer questions using private documents.
Enterprise Search	Search across internal data (PDFs, DBs, etc).
Document Summarization	Summarize large documents.
Code Assistant	Generate, explain and review code.
Data Analysis	Query databases using natural language.
Customer Support	Automate support using LLM + knowledge base.
Multimodal Applications	Work with text, images, audio, video.

3. End-to-End RAG System Design



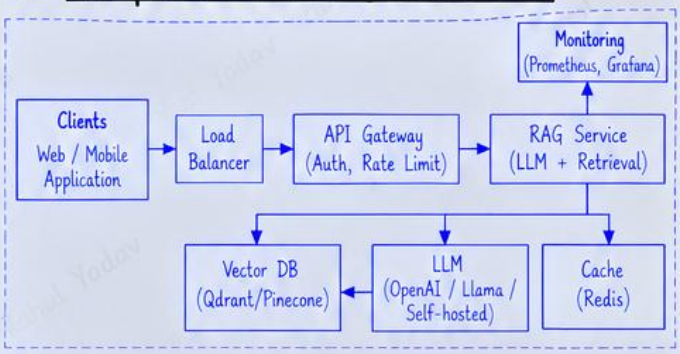
4. Key Components and Responsibilities

Component	Responsibility
Data Ingestion	Load and preprocess documents (PDF, web, DB).
Text Splitting	Split documents into smaller chunks.
Embedding Model	Convert text to vector embeddings.
Vector Database	Store and search embeddings (Qdrant, Pinecone).
Retriever	Find relevant chunks for the query.
Reranker (Optional)	Re-rank results for better relevance.
LLM	Generate final answer using retrieved context.
Guardrails	Safety checks, prompt filtering, output validation.
Caching	Cache responses to reduce latency and cost.
Monitoring	Track performance, errors and user feedback.

5. Scaling Strategies

- Use distributed vector database (Pinecone, Qdrant cluster).
- Implement caching (Redis) for frequent queries.
- Use asynchronous processing for heavy tasks.
- Use load balancers and auto-scaling (Kubernetes).
- Optimize embeddings and chunk size.
- Use batch processing for large document ingestion.
- Monitor and scale based on traffic and latency.

6. Example: Scalable RAG Architecture



7. Design Considerations

Aspect	Questions to Consider
Scalability	How to handle high traffic and large data ?
Latency	How to reduce response time ?
Cost	How to optimize model usage and infrastructure cost ?
Security	How to protect data and prevent prompt injection ?
Data Privacy	How to handle sensitive enterprise data ?
Reliability	How to ensure high availability and fault tolerance ?
Monitoring	What metrics and logs to track ?
Model Updates	How to deploy new model versions without downtime ?

8. Common Interview Questions

- Design a RAG system for an enterprise knowledge base.
- How will you handle scaling for millions of documents ?
- How do you reduce hallucinations in LLM responses ?
- How do you optimize cost for high volume queries ?
- How do you ensure data security and prevent prompt injection ?
- What monitoring and metrics will you track in production ?
- How will you handle multi-modal data (text + image) ?
- How do you choose between OpenAI, Claude, Llama or self-hosted models ?
- How do you design a chatbot with memory ?
- What are the trade-offs between accuracy, latency and cost ?
- How will you handle model versioning and rollback ?

9. Real-World Challenges and Solutions

Challenge	Solution
Large document size	Chunking, hierarchical indexing, summarization.
Irrelevant search results	Better embeddings, reranking, hybrid search.
High latency	Caching, optimized prompts, smaller models.
High cost	Use open-source models, caching, batch processing.
Hallucinations	RAG, citations, guardrails, prompt engineering.
Security risks	Input validation, access control, content filtering.
Scalability	Distributed systems, auto-scaling, load balancing.
Model failures	Fallback models, retries, monitoring and alerts.
Data updates	Incremental indexing, versioning, re-embedding.
User personalization	Session memory, user profiles, contextual retrieval.

10. Key Takeaways

- System design is essential for building production-ready LLM applications.
- Focus on scalability, cost, security and reliability.
- Understand trade-offs and real-world challenges.
- Practice system design and common interview questions.
- Use best practices and monitoring for continuous improvement.

Rahul Yadav

19. AI Engineering Interview Questions and Final Revision

1. Common Interview Topics

- Core AI/ML concepts (supervised, unsupervised, deep learning).
- LLMs and transformer architecture.
- Prompt engineering and RAG.
- Vector databases (Qdrant, Pinecone, FAISS).
- LangChain, LlamaIndex, agents and tool use.
- Model serving, deployment and scaling.
- MLOps, monitoring and evaluation.
- System design for real-world AI applications.
- Coding and problem solving (Python, data structures).
- Behavioral and scenario-based questions.

2. LLM Conceptual Questions

Question	Key Points to Answer
What are LLMs ?	Definition, pretraining, capabilities, use cases.
How do transformers work ?	Attention, encoder-decoder, self-attention.
What is the difference between BERT and GPT ?	Architecture, training objective, use cases.
What is tokenization ?	Tokenizers, subword units, BPE, SentencePiece.
What are embeddings ?	Vector representation, semantic similarity.
What is context window ?	Maximum tokens the model can consider.
What is hallucination ?	Definition, causes, mitigation techniques.
What is RAG ?	Retrieve relevant context to ground responses.

3. Prompt Engineering Questions

Question	What to Cover
How do you write good prompts ?	Clear instructions, context, examples, output format.
What are prompt techniques ?	Zero-shot, few-shot, chain-of-thought, self-consistency.
How do you reduce hallucinations ?	Provide context, use RAG, structured output, validation, guardrails.
How do you handle long context ?	Chunking, summarization, map-reduce, hierarchical retrieval.

4. Vector Database Questions

Question	What to Cover
What is a vector database ?	Store and search vector embeddings.
How does similarity search work ?	Cosine similarity, Euclidean distance, ANN.
Qdrant vs Pinecone vs FAISS ?	Architecture, deployment, features, use cases.
How do you design embedding pipeline ?	Chunking, embedding model, metadata, storage, retrieval.

5. System Design Questions (Common)

Question	Key Points
Design a RAG system	Data ingestion, chunking, embeddings, vector DB, retrieval, LLM, caching, monitoring.
Design a chatbot for enterprise	Authentication, data security, RAG, tools, evaluation, deployment.
Design a real-time document QA system	Ingestion pipeline, processing, retrieval, low latency.
Design a multi-agent system	Agent roles, tool use, orchestration, memory, evaluation.
Design for scalability	Load balancing, caching, queue, autoscaling, observability.

6. MLOps and Deployment Questions

Question	What to Cover
How do you deploy an LLM ?	Model serving, API, containerization, scaling.
How do you monitor an LLM ?	Latency, throughput, errors, quality, cost.
How do you handle model updates ?	Versioning, A/B testing, rollback.
How do you ensure reliability ?	Health checks, retries, fallback models.
How do you reduce cost ?	Caching, smaller models, quantization, optimized infrastructure.

7. Coding / Practical Questions

Problem	Key Concepts
Build a simple RAG pipeline in Python	Document loading, chunking, embeddings, vector search, LLM query.
Implement text chunking	RecursiveCharacterTextSplitter or custom logic.
Similarity search using FAISS	Create index, add vectors, search top k.
Call OpenAI API	API key, request/response, error handling.
Build a FastAPI endpoint for LLM	POST API, request validation, streaming response.
Implement caching for LLM responses	Redis, in-memory cache, cache key strategy.

8. Behavioral and Scenario Questions

Question	How to Approach
How do you choose the right model for a use case ?	Consider accuracy, latency, cost, data privacy.
How do you handle sensitive data in LLM applications ?	Data anonymization, access control, on-prem models, compliance.
What trade-offs do you consider in production ?	Performance vs cost vs accuracy vs scalability.
How do you work with cross-functional teams ?	Clear communication, requirement analysis, iterative delivery.
Tell me about a challenging problem you solved.	Use STAR format (Situation, Task, Action, Result).

9. Final Revision Checklist

- ☑ Revise all key concepts, diagrams and architectures.
- ☑ Practice common interview questions.
- ☑ Implement small hands-on projects (RAG, chatbot, API).
- ☑ Understand trade-offs and treal-world challenges.
- ☑ Revise important tools and frameworks (LangChain, LlamaIndex, Qdrant).
- ☑ Practice system design problems.
- ☑ Review MLOps and deployment concepts.
- ☑ Be ready for coding and practical implementation questions.
- ☑ Focus on explaining your past projects with clear problem, approach and impact.

10. Key Takeaways

- Understand fundamentals deeply, not just definitions.
- Be able to explain concepts with real-world examples.
- Practice both theory and hands-on implementation.
- Focus on system design and trade-offs.
- Stay updated with latest tools and best practices.
- Be confident and structured in your answers.
- Show problem-solving skills and clear communication.
- Learn from past projects and apply them to interview scenarios.

Rahul Yadav